

```

inline MyClass::MyClass() :
    MyClass(impl::call_factory<MyClass>([](auto&& f){
        return f.template ActivateInstance<MyClass>(); })))
{
}
inline void MyClass::StaticMethod()
{
    impl::call_factory<MyClass, MyProject::IClassStatics>([&](auto&& f)
{
        return f.StaticMethod(); }));
}
}

```

It's not necessary to follow all of the above; the intent is to show that both calls involve a call to a function named `call_factory`. That's your clue that these calls involve the factory cache, and they're not directly accessing the implementation. With `-opt[imize]`, these same functions aren't defined at all. Instead, they're declared by the projection, and their definitions are left up to the component.

The component can then provide definitions that call directly into the implementation. Now we've arrived at the breaking change. Those definitions are generated for you when you use both `-component` and `-opt[imize]`, and they appear in a file named `Type.g.cpp`, where `Type` is the name of the runtime class being implemented. That's why you may hit various linker errors when you first enable `-opt[imize]` in an existing project. You need to include that generated file into your implementation in order to stitch things up.

In our example, `MyClass.h` might look like this (regardless of whether `-opt[imize]` is being used).

C++/WinRT

```

// MyClass.h
#pragma once
#include "MyClass.g.h"

namespace winrt::MyProject::implementation
{
    struct MyClass : ClassT<MyClass>
    {
        MyClass() = default;

        static void StaticMethod();
        void Method();
    };
}
namespace winrt::MyProject::factory_implementation
{
    struct MyClass : ClassT<MyClass, implementation::MyClass>

```

```

    {
    };
}

```

Your `MyClass.cpp` is where it all comes together.

C++/WinRT

```

#include "pch.h"
#include "MyClass.h"
#include "MyClass.g.cpp" // !!It's important that you add this line!!

namespace winrt::MyProject::implementation
{
    void MyClass::StaticMethod()
    {
    }

    void MyClass::Method()
    {
    }
}

```

So, to use uniform construction in an existing project, you need to edit each implementation's `.cpp` file so that you `#include <Sub/Namespace/Type.g.cpp>` after the inclusion (and definition) of the implementation class. That file provides the definitions of those functions that the projection left undefined. Here's what those definitions look like inside the `MyClass.g.cpp` file.

C++/WinRT

```

namespace winrt::MyProject
{
    MyClass::MyClass() :
        MyClass(make<MyProject::implementation::MyClass>())
    {
    }
    void MyClass::StaticMethod()
    {
        return MyProject::implementation::MyClass::StaticMethod();
    }
}

```

And that nicely completes the projection with efficient calls directly into the implementation, avoiding calls to the factory cache, and satisfying the linker.

The final thing that `-opt[imize]` does for you is to change the implementation of your project's `module.g.cpp` (the file that helps you to implement your DLL's

`DllGetActivationFactory` and `DllCanUnloadNow` exports) in such a way that incremental builds will tend to be much faster by eliminating the strong type coupling that was required by C++/WinRT 1.0. This is often referred to as *type-erased factories*. Without `-opt[imize]`, the `module.g.cpp` file that's generated for your component starts off by including the definitions of all of your implementation classes—the `MyClass.h`, in this example. It then directly creates the implementation factory for each class like this.

C++/WinRT

```
if (requal(name, L"MyProject.MyClass"))
{
    return
    winrt::detach_abi(winrt::make<winrt::MyProject::factory_implementation::MyClass>());
}
```

Again, you don't need to follow all of the details. What's useful to see is that this requires the complete definition for any and all classes implemented by your component. This can have a dramatic effect on your inner loop, as any change to a single implementation will cause `module.g.cpp` to recompile. With `-opt[imize]`, this is no longer the case. Instead, two things happen to the generated `module.g.cpp` file. The first is that it no longer includes any implementation classes. In this example, it won't include `MyClass.h` at all. Instead, it creates the implementation factories without any knowledge of their implementation.

C++/WinRT

```
void* winrt_make_MyProject_MyClass();

if (requal(name, L"MyProject.MyClass"))
{
    return winrt_make_MyProject_MyClass();
}
```

Obviously, there's no need to include their definitions, and it's up to the linker to resolve the `winrt_make_Component_Class` function's definition. Of course, you don't need to think about this, because the `MyClass.g.cpp` file that gets generated for you (and that you previously included in order to support uniform construction) also defines this function. Here's the entirety of the `MyClass.g.cpp` file that's generated for this example.

C++/WinRT

```
void* winrt_make_MyProject_MyClass()
{
```

```

        return
winrt::detach_abi(winrt::make<winrt::MyProject::factory_implementation::MyCl
ass>());
}
namespace winrt::MyProject
{
    MyClass::MyClass() :
        MyClass(make<MyProject::implementation::MyClass>())
    {
    }
    void MyClass::StaticMethod()
    {
        return MyProject::implementation::MyClass::StaticMethod();
    }
}

```

As you can see, the `winrt_make_MyProject_MyClass` function directly creates your implementation's factory. This all means that you can happily change any given implementation, and the `module.g.cpp` needn't be recompiled at all. It's only when you add or remove Windows Runtime classes that `module.g.cpp` will be updated, and need to be recompiled.

Overriding base class virtual methods

Your derived class can have issues with virtual methods if both the base and the derived class are app-defined classes, but the virtual method is defined in a grandparent Windows Runtime class. In practice, this happens if you derive from XAML classes. The rest of this section continues from the example in [Derived classes](#).

C++/WinRT

```

namespace winrt::MyNamespace::implementation
{
    struct BasePage : BasePageT<BasePage>
    {
        void
        OnNavigatedFrom(Windows::UI::Xaml::Navigation::NavigationEventArgs const&
e);
    };

    struct DerivedPage : DerivedPageT<DerivedPage>
    {
        void
        OnNavigatedFrom(Windows::UI::Xaml::Navigation::NavigationEventArgs const&
e);
    };
}

```

The hierarchy is `Windows::UI::Xaml::Controls::Page` <- `BasePage` <- `DerivedPage`. The `BasePage::OnNavigatedFrom` method correctly overrides `Page::OnNavigatedFrom`, but `DerivedPage::OnNavigatedFrom` doesn't override `BasePage::OnNavigatedFrom`.

Here, `DerivedPage` reuses the `IPageOverrides` vtable from `BasePage`, which means that it fails to override the `IPageOverrides::OnNavigatedFrom` method. One potential solution requires `BasePage` to itself be a template class, and to have its implementation entirely in a header file, but that makes things unacceptably complicated.

As a workaround, declare the `OnNavigatedFrom` method as explicitly virtual in the base class. That way, when the vtable entry for

`DerivedPage::IPageOverrides::OnNavigatedFrom` calls

`BasePage::IPageOverrides::OnNavigatedFrom`, the producer calls

`BasePage::OnNavigatedFrom`, which (due to its virtualness), ends up calling

`DerivedPage::OnNavigatedFrom`.

C++/WinRT

```
namespace winrt::MyNamespace::implementation
{
    struct BasePage : BasePageT<BasePage>
    {
        // Note the `virtual` keyword here.
        virtual void
        OnNavigatedFrom(Windows::UI::Xaml::Navigation::NavigationEventArgs const&
            e);
    };

    struct DerivedPage : DerivedPageT<DerivedPage>
    {
        void
        OnNavigatedFrom(Windows::UI::Xaml::Navigation::NavigationEventArgs const&
            e);
    };
}
```

This requires that all members of the class hierarchy agree on the return value and parameter types of the `OnNavigatedFrom` method. If they disagree, then you should use the version above as the virtual method, and wrap the alternates.

⚠ Note

Your IDL doesn't need to declare the overridden method. For more details, see [Implementing overridable methods](#).

Important APIs

- [winrt::com_ptr struct template](#)
- [winrt::com_ptr::copy_from function](#)
- [winrt::from_abi function template](#)
- [winrt::get_self function template](#)
- [winrt::implements struct template](#)
- [winrt::make function template](#)
- [winrt::make_self function template](#)
- [winrt::Windows::Foundation::IUnknown::as function](#)
- [winrt::Windows::Foundation::IUnknown::try_as function](#)

Related topics

- [Author events in C++/WinRT](#)
- [Consume APIs with C++/WinRT](#)
- [Windows Runtime components with C++/WinRT](#)
- [XAML controls; bind to a C++/WinRT property](#)

Feedback

Was this page helpful?



[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Error handling with C++/WinRT

Article • 10/20/2022

This topic discusses strategies for handling errors when programming with [C++/WinRT](#). For more general info, and background, see [Errors and Exception Handling \(Modern C++\)](#).

Avoid catching and throwing exceptions

We recommend that you continue to write [exception-safe code](#), but that you prefer to avoid catching and throwing exceptions whenever possible. If there's no handler for an exception, then Windows automatically generates an error report (including a minidump of the crash), which will help you track down where the problem is.

Don't throw an exception that you expect to catch. And don't use exceptions for expected failures. Throw an exception *only when an unexpected runtime error occurs*, and handle everything else with error/result codes—directly, and close to the source of the failure. That way, when an exception is thrown, you know that the cause is either a bug in your code, or an exceptional error state in the system.

Consider the scenario of accessing the Windows Registry. If your app fails to read a value from the Registry, then that's to be expected, and you should handle it gracefully. Don't throw an exception; rather return a `bool` or `enum` value indicating that, and perhaps why, the value wasn't read. Failing to *write* a value to the Registry, on the other hand, is likely to indicate that there's a bigger problem than you can handle sensibly in your application. In a case like that, you don't want your application to continue, so an exception that results in an error report is the fastest way to keep your application from causing any harm.

For another example, consider retrieving a thumbnail image from a call to [StorageFile.GetThumbnailAsync](#), and then passing that thumbnail to [BitmapSource.SetSourceAsync](#). If that sequence of calls causes you to pass `nullptr` to [SetSourceAsync](#) (the image file can't be read; perhaps its file extension makes it look like it contains image data, but it actually doesn't), then you'll cause an invalid pointer exception to be thrown. If you discover a case like that in your code, rather than catching and handling the case as an exception, instead check for `nullptr` returned from [GetThumbnailAsync](#).

Throwing exceptions tends to be slower than using error codes. If you only throw an exception when a fatal error occurs, then if all goes well you'll never pay the

performance price.

But a more likely performance hit involves the runtime overhead of ensuring that the appropriate destructors are called in the unlikely event that an exception is thrown. The cost of this assurance comes whether an exception is actually thrown or not. So, you should ensure that the compiler has a good idea of what functions can potentially throw exceptions. If the compiler can prove that there won't be any exceptions from certain functions (the `noexcept` specification), then it can optimize the code it generates.

Catching exceptions

An error condition that arises at the [Windows Runtime ABI](#) layer is returned in the form of a HRESULT value. But you don't need to handle HRESULTs in your code. The C++/WinRT projection code that's generated for an API on the consuming side detects an error HRESULT code at the ABI layer and converts the code into a [winrt::hresult_error](#) exception, which you can catch and handle. If you *do* wish to handle HRESULTS, then use the `winrt::hresult` type.

For example, if the user happens to delete an image from the Pictures Library while your application is iterating over that collection, then the projection throws an exception. And this is a case where you'll have to catch and handle that exception. Here's a code example showing this case.

C++/WinRT

```
#include <winrt/Windows.Foundation.Collections.h>
#include <winrt/Windows.Storage.h>
#include <winrt/Windows.UI.Xaml.Media.Imaging.h>

using namespace winrt;
using namespace Windows::Foundation;
using namespace Windows::Storage;
using namespace Windows::UI::Xaml::Media::Imaging;

IAsyncAction MakeThumbnailsAsync()
{
    auto imageFiles{ co_await
KnownFolders::PicturesLibrary().GetFilesAsync() };

    for (StorageFile const& imageFile : imageFiles)
    {
        BitmapImage bitmapImage;
        try
        {
            auto thumbnail{ co_await
imageFile.GetThumbnailAsync(FileProperties::ThumbnailMode::PicturesView) };
            if (thumbnail) bitmapImage.SetSource(thumbnail);
        }
    }
}
```

```

    }
    catch (winrt::hresult_error const& ex)
    {
        winrt::hresult hr = ex.code(); //
        HRESULT_FROM_WIN32(ERROR_FILE_NOT_FOUND).
        winrt::hstring message = ex.message(); // The system cannot find
        the file specified.
    }
}
}

```

Use this same pattern in a coroutine when calling a `co_await`-ed function. Another example of this HRESULT-to-exception conversion is that when a component API returns `E_OUTOFMEMORY`, that causes a `std::bad_alloc` to be thrown.

Prefer `winrt::hresult_error::code` when you're just peeking at a HRESULT code. The `winrt::hresult_error::to_abi` function on the other hand converts to a COM error object, and pushes state into the COM thread-local storage.

Throwing exceptions

There will be cases where you decide that, should your call to a given function fail, your application won't be able to recover (you'll no longer be able to rely on it to function predictably). The code example below uses a `winrt::handle` value as a wrapper around the `HANDLE` returned from `CreateEvent`. It then passes the handle (creating a `bool` value from it) to the `winrt::check_bool` function template. `winrt::check_bool` works with a `bool`, or with any value that's convertible to `false` (an error condition), or `true` (a success condition).

C++/WinRT

```

winrt::handle h{ ::CreateEvent(nullptr, false, false, nullptr) };
winrt::check_bool(bool{ h });
winrt::check_bool(::SetEvent(h.get()));

```

If the value that you pass to `winrt::check_bool` is false, then the following sequence of actions take place.

- `winrt::check_bool` calls the `winrt::throw_last_error` function.
- `winrt::throw_last_error` calls `GetLastError` to retrieve the calling thread's last-error code value, and then calls the `winrt::throw_hresult` function.
- `winrt::throw_hresult` throws an exception using a `winrt::hresult_error` object (or a standard object) that represents that error code.

Because Windows APIs report run-time errors using various return-value types, there are in addition to `winrt::check_bool` a handful of other useful helper functions for checking values and throwing exceptions.

- `winrt::check_hresult`. Checks whether the HRESULT code represents an error and, if so, calls `winrt::throw_hresult`.
- `winrt::check_nt`. Checks whether a code represents an error and, if so, calls `winrt::throw_hresult`.
- `winrt::check_pointer`. Checks whether a pointer is null and, if so, calls `winrt::throw_last_error`.
- `winrt::check_win32`. Checks whether a code represents an error and, if so, calls `winrt::throw_hresult`.

You can use these helper functions for common return code types, or you can respond to any error condition and call either `winrt::throw_last_error` or `winrt::throw_hresult`.

Throwing exceptions when authoring an API

All [Windows Runtime Application Binary Interface](#) boundaries (or ABI boundaries) must be *noexcept*—meaning that exceptions must never escape there. When you author an API, you should always mark the ABI boundary with the C++ `noexcept` keyword.

`noexcept` has specific behavior in C++. If a C++ exception hits a `noexcept` boundary, then the process will fail fast with `std::terminate`. That behavior is generally desirable, because an unhandled exception almost always implies unknown state in the process.

Since exceptions mustn't cross the ABI boundary, an error condition that arises in an implementation is returned across the ABI layer in the form of an HRESULT error code. When you're authoring an API using C++/WinRT, code is generated for you to convert any exception that you *do* throw in your implementation into an HRESULT. The `winrt::to_hresult` function is used in that generated code in a pattern like this.

C++/WinRT

```
HRESULT DoWork() noexcept
{
    try
    {
        // Shim through to your C++/WinRT implementation.
        return S_OK;
    }
    catch (...)
    {
        return winrt::to_hresult(); // Convert any exception to an HRESULT.
    }
}
```

`winrt::to_hresult` handles exceptions derived from `std::exception`, and `winrt::hresult_error` and its derived types. In your implementation, you should prefer `winrt::hresult_error`, or a derived type, so that consumers of your API receive rich error information. `std::exception` (which maps to `E_FAIL`) is supported in case exceptions arise from your use of the Standard Template Library.

Debuggability with `noexcept`

As we mentioned above, a C++ exception hitting a `noexcept` boundary fails fast with `std::terminate`. That's not ideal for debugging, because `std::terminate` often loses much or all of the error or the exception context thrown, especially when coroutines are involved.

So, this section deals with the case where your ABI method (which you've properly annotated with `noexcept`) uses `co_await` to call asynchronous C++/WinRT projection code. We recommend that you wrap the calls to the C++/WinRT projection code within a `winrt::fire_and_forget`. Doing so provides a proper place for an unhandled exception to be properly recorded as a stowed exception, which greatly increases debuggability.

C++/WinRT

```
HRESULT MyWinRTObject::MyABI_Method() noexcept
{
    winrt::com_ptr<Foo> foo{ get_a_foo() };

    [/*no captures*/](winrt::com_ptr<Foo> foo) -> winrt::fire_and_forget
    {
        co_await winrt::resume_background();

        foo->ABICall();

        AnotherMethodWithLotsOfProjectionCalls();
    }(foo);

    return S_OK;
}
```

`winrt::fire_and_forget` has a built-in `unhandled_exception` method helper, which calls `winrt::terminate`, which in turn calls `RoFailFastWithErrorContext`. This guarantees that any context (stowed exception, error code, error message, stack backtrace, and so on) is preserved either for live debugging or for a post-mortem dump. For convenience, you can factor the fire-and-forget portion into a separate function that returns a `winrt::fire_and_forget`, and then call that.

Synchronous code

In some cases, your ABI method (which, again, you've properly annotated with `noexcept`) calls only synchronous code. In other words, it never uses `co_await`, either to call an asynchronous Windows Runtime method, or to switch between foreground and background threads. In that case, the `fire_and_forget` technique will still work, but it's not efficient. Instead, you can do something like this.

C++/WinRT

```
HRESULT abi() noexcept try
{
    // ABI code goes here.
} catch (...) { winrt::terminate(); }
```

Fail fast

The code in the previous section still fails fast. As written, that code doesn't handle any exceptions. Any unhandled exception results in program termination.

But that form is superior, because it ensures debuggability. In rare cases, you might want to `try/catch`, and handle certain exceptions. But that should be rare because, as this topic explains, we discourage using exceptions as a flow-control mechanism for conditions that you expect.

Remember that it's a bad idea to let an unhandled exception escape a naked `noexcept` context. Under that condition, the C++ runtime will **`std::terminate`** the process, thereby losing any stowed exception information that C++/WinRT carefully recorded.

Assertions

For internal assumptions in your application, there are assertions. Prefer **`static_assert`** for compile-time validation, wherever possible. For run-time conditions, use `WINRT_ASSERT` with a Boolean expression. `WINRT_ASSERT` is a macro definition, and it expands to [`\_ASSERTE`](#).

C++/WinRT

```
WINRT_ASSERT(pos < size());
```

`WINRT_ASSERT` is compiled away in release builds; in a debug build, it stops the application in the debugger on the line of code where the assertion is.

You shouldn't use exceptions in your destructors. So, at least in debug builds, you can assert the result of calling a function from a destructor with `WINRT_VERIFY` (with a Boolean expression) and `WINRT_VERIFY_` (with an expected result and a Boolean expression).

C++/WinRT

```
WINRT_VERIFY(::CloseHandle(value));  
WINRT_VERIFY_(TRUE, ::CloseHandle(value));
```

Important APIs

- [winrt::check_bool](#) function template
- [winrt::check_hresult](#) function
- [winrt::check_nt](#) function template
- [winrt::check_pointer](#) function template
- [winrt::check_win32](#) function template
- [winrt::handle](#) struct
- [winrt::hresult_error](#) struct
- [winrt::throw_hresult](#) function
- [winrt::throw_last_error](#) function
- [winrt::to_hresult](#) function

Related topics

- [Errors and Exception Handling \(Modern C++\)](#)
- [How to: Design for Exception Safety](#)

Handle events by using delegates in C++/WinRT

Article • 10/20/2022

This topic shows how to register and revoke event-handling delegates using [C++/WinRT](#). You can handle an event using any standard C++ function-like object.

ⓘ Note

For info about installing and using the C++/WinRT Visual Studio Extension (VSIX) and the NuGet package (which together provide project template and build support), see [Visual Studio support for C++/WinRT](#).

Using Visual Studio to add an event handler

A convenient way of adding an event handler to your project is by using the XAML Designer user interface (UI) in Visual Studio. With your XAML page open in the XAML Designer, select the control whose event you want to handle. Over in the property page for that control, click on the lightning-bolt icon to list all of the events that are sourced by that control. Then, double-click on the event that you want to handle; for example, *OnClicked*.

The XAML Designer adds the appropriate event handler function prototype (and a stub implementation) to your source files, ready for you to replace with your own implementation.

ⓘ Note

Typically, your event handlers don't need to be described in your Midl file (`.idl`). So, the XAML Designer doesn't add event handler function prototypes to your Midl file. It only adds them your `.h` and `.cpp` files.

Register a delegate to handle an event

A simple example is handling a button's click event. It's typical to use XAML markup to register a member function to handle the event, like this.

XAML

```
// MainPage.xaml
<Button x:Name="myButton" Click="ClickHandler">Click Me</Button>
```

C++/WinRT

```
// MainPage.h
void ClickHandler(
    winrt::Windows::Foundation::IInspectable const& sender,
    winrt::Windows::UI::Xaml::RoutedEventArgs const& args);

// MainPage.cpp
void MainPage::ClickHandler(
    IInspectable const& /* sender */,
    RoutedEventArgs const& /* args */)
{
    myButton().Content(box_value(L"Clicked"));
}
```

The code above is taken from the **Blank App (C++/WinRT)** project in Visual Studio. The code `myButton()` calls a generated accessor function, which returns the **Button** that we named *myButton*. If you change the `x:Name` of that **Button** element, then the name of the generated accessor function changes, too.

❗ Note

In this case, the event source (the object that raises the event) is the **Button** named *myButton*. And the event recipient (the object handling the event) is an instance of **MainPage**. There's more info later in this topic about managing the lifetime of event sources and event recipients.

Instead of doing it declaratively in markup, you can imperatively register a member function to handle an event. It may not be obvious from the code example below, but the argument to the `ButtonBase::Click` call is an instance of the **RoutedEventHandler** delegate. In this case, we're using the **RoutedEventHandler** constructor overload that takes an object and a pointer-to-member-function.

C++/WinRT

```
// MainPage.cpp
MainPage::MainPage()
{
    InitializeComponent();
}
```

```
myButton().Click({ this, &MainPage::ClickHandler });
}
```

❗ Important

When registering the delegate, the code example above passes a raw *this* pointer (pointing to the current object). To learn how to establish a strong or a weak reference to the current object, see [If you use a member function as a delegate](#).

Here's an example that uses a static member function; note the simpler syntax.

C++/WinRT

```
// MainPage.h
static void ClickHandler(
    winrt::Windows::Foundation::IInspectable const& sender,
    winrt::Windows::UI::Xaml::RoutedEventArgs const& args);

// MainPage.cpp
MainPage::MainPage()
{
    InitializeComponent();

    myButton().Click( MainPage::ClickHandler );
}
void MainPage::ClickHandler(
    IInspectable const& /* sender */,
    RoutedEventArgs const& /* args */) { ... }
```

There are other ways to construct a **RoutedEventHandler**. Below is the syntax block taken from the documentation topic for [RoutedEventHandler](#) (choose C++/WinRT from the **Language** drop-down in the upper-right corner of the webpage). Notice the various constructors: one takes a lambda; another a free function; and another (the one we used above) takes an object and a pointer-to-member-function.

C++/WinRT

```
struct RoutedEventHandler : winrt::Windows::Foundation::IUnknown
{
    RoutedEventHandler(std::nullptr_t = nullptr) noexcept;
    template <typename L> RoutedEventHandler(L lambda);
    template <typename F> RoutedEventHandler(F* function);
    template <typename O, typename M> RoutedEventHandler(O* object, M
method);
    /* ... other constructors ... */
    void operator()(winrt::Windows::Foundation::IInspectable const& sender,
```

```
winrt::Windows::UI::Xaml::RoutedEventArgs const& e) const;
};
```

The syntax of the function call operator is also helpful to see. It tells you what your delegate's parameters need to be. As you can see, in this case the function call operator syntax matches the parameters of our **MainPage::ClickHandler**.

ⓘ Note

For any given event, to figure out the details of its delegate, and that delegate's parameters, go first to the documentation topic for the event itself. Let's take the **UIElement.KeyDown event** as an example. Visit that topic, and choose **C++/WinRT** from the **Language** drop-down. In the syntax block at the beginning of the topic, you'll see this.

C++/WinRT

```
// Register
event_token KeyDown(KeyEventHandler const& handler) const;
```

That info tells us that the **UIElement.KeyDown** event (the topic we're on) has a delegate type of **KeyEventHandler**, since that's the type that you pass when you register a delegate with this event type. So, now follow the link on the topic to that **KeyEventHandler delegate** type. Here, the syntax block contains a function call operator. And, as mentioned above, that tells you what your delegate's parameters need to be.

C++/WinRT

```
void operator()(
    winrt::Windows::Foundation::IInspectable const& sender,
    winrt::Windows::UI::Xaml::Input::KeyRoutedEventArgs const& e) const;
```

As you can see, the delegate needs to be declared to take an **IInspectable** as the sender, and an instance of the **KeyRoutedEventArgs** class as the args.

To take another example, let's look at the **Popup.Closed** event. Its delegate type is **EventHandler<IInspectable>**. So, your delegate will take an **IInspectable** as the sender, and another **IInspectable** (because that's the **EventHandler**'s type parameter) as the args.

If you're not doing much work in your event handler, then you can use a lambda function instead of a member function. Again, it may not be obvious from the code example below, but a **RoutedEventHandler** delegate is being constructed from a lambda function which, again, needs to match the syntax of the function call operator that we discussed above.

C++/WinRT

```
MainPage::MainPage()
{
    InitializeComponent();

    myButton().Click([this](IInspectable const& /* sender */,
RoutedEventArgs const& /* args */)
    {
        myButton().Content(box_value(L"Clicked"));
    });
}
```

You can choose to be a little more explicit when you construct your delegate. For example, if you want to pass it around, or use it more than once.

C++/WinRT

```
MainPage::MainPage()
{
    InitializeComponent();

    auto click_handler = [](IInspectable const& sender, RoutedEventArgs
const& /* args */)
    {
        sender.as<winrt::Windows::UI::Xaml::Controls::Button>
().Content(box_value(L"Clicked"));
    };
    myButton().Click(click_handler);
    AnotherButton().Click(click_handler);
}
```

Revoke a registered delegate

When you register a delegate, typically a token is returned to you. You can subsequently use that token to revoke your delegate; meaning that the delegate is unregistered from the event, and won't be called should the event be raised again.

For the sake of simplicity, none of the code examples above showed how to do that. But this next code example stores the token in the struct's private data member, and

revokes its handler in the destructor.

C++/WinRT

```
struct Example : ExampleT<Example>
{
    Example(winrt::Windows::UI::Xaml::Controls::Button const& button) :
    m_button(button)
    {
        m_token = m_button.Click([this](IInspectable const&, RoutedEventArgs
const&))
        {
            // ...
        });
    }
    ~Example()
    {
        m_button.Click(m_token);
    }

private:
    winrt::Windows::UI::Xaml::Controls::Button m_button;
    winrt::event_token m_token;
};
```

Instead of a strong reference, as in the example above, you can store a weak reference to the button (see [Strong and weak references in C++/WinRT](#)).

ⓘ Note

When an event source raises its events synchronously, you can revoke your handler and be confident that you won't receive any more events. But for asynchronous events, even after revoking (and especially when revoking within the destructor), an in-flight event might reach your object after it has started destructing. Finding a place to unsubscribe prior to destruction might mitigate the issue, or for a robust solution see [Safely accessing the *this* pointer with an event-handling delegate](#).

Alternatively, when you register a delegate, you can specify `winrt::auto_revoke` (which is a value of type `winrt::auto_revoke_t`) to request an event revoker (of type `winrt::event_revoker`). The event revoker holds a weak reference to the event source (the object that raises the event) for you. You can manually revoke by calling the `event_revoker::revoke` member function; but the event revoker calls that function itself automatically when it goes out of scope. The `revoke` function checks whether the event source still exists and, if so, revokes your delegate. In this example, there's no need to store the event source, and no need for a destructor.

C++/WinRT

```
struct Example : ExampleT<Example>
{
    Example(winrt::Windows::UI::Xaml::Controls::Button button)
    {
        m_event_revoker = button.Click(
            winrt::auto_revoke,
            [this](IInspectable const& /* sender */,
                RoutedEventArgs const& /* args */)
            {
                // ...
            });
    }

private:
    winrt::Windows::UI::Xaml::Controls::Button::Click_revoker
    m_event_revoker;
};
```

Below is the syntax block taken from the documentation topic for the [ButtonBase::Click](#) event. It shows the three different registration and revoking functions. You can see exactly what type of event revoker you need to declare from the third overload. And you can pass the same kinds of delegates to both the *register* and the *revoke with event_revoker* overloads.

C++/WinRT

```
// Register
winrt::event_token Click(winrt::Windows::UI::Xaml::RoutedEventHandler const&
    handler) const;

// Revoke with event_token
void Click(winrt::event_token const& token) const;

// Revoke with event_revoker
Button::Click_revoker Click(winrt::auto_revoke_t,
    winrt::Windows::UI::Xaml::RoutedEventHandler const& handler) const;
```

ⓘ Note

In the code example above, `Button::Click_revoker` is a type alias for `winrt::event_revoker<winrt::Windows::UI::Xaml::Controls::Primitives::IButtonBase>`. A similar pattern applies to all C++/WinRT events. Each Windows Runtime event has a revoke function overload that returns an event revoker, and that revoker's type is a member of the event source. So, to take another example, the

CoreApplication::SizeChanged event has a registration function overload that returns a value of type **CoreApplication::SizeChanged_revoker**.

You might consider revoking handlers in a page-navigation scenario. If you're repeatedly navigating into a page and then back out, then you could revoke any handlers when you navigate away from the page. Alternatively, if you're re-using the same page instance, then check the value of your token and only register if it's not yet been set (`if (!m_token){ ... }`). A third option is to store an event revoker in the page as a data member. And a fourth option, as described later in this topic, is to capture a strong or a weak reference to the *this* object in your lambda function.

If your auto-revoke delegate fails to register

If you try to specify **winrt::auto_revoke** when registering a delegate, and the result is a **winrt::hresult_no_interface** exception, then that usually means that the event source doesn't support weak references. That's a common situation in the **Windows.UI.Composition** namespace, for example. In this situation, you can't use the auto-revoke feature. You'll have to fall back to manually revoking your event handlers.

Delegate types for asynchronous actions and operations

The examples above use the **RoutedEventHandler** delegate type, but there are of course many other delegate types. For example, asynchronous actions and operations (with and without progress) have completed and/or progress events that expect delegates of the corresponding type. For example, the progress event of an asynchronous operation with progress (which is anything that implements **IAsyncOperationWithProgress**) requires a delegate of type **AsyncOperationProgressHandler**. Here's a code example of authoring a delegate of that type using a lambda function. The example also shows how to author an **AsyncOperationWithProgressCompletedHandler** delegate.

C++/WinRT

```
#include <winrt/Windows.Foundation.h>
#include <winrt/Windows.Web.Syndication.h>

using namespace winrt;
using namespace Windows::Foundation;
using namespace Windows::Web::Syndication;

void ProcessFeedAsync()
{
```

```

Uri rssFeedUri{ L"https://blogs.windows.com/feed" };
SyndicationClient syndicationClient;

auto async_op_with_progress =
syndicationClient.RetrieveFeedAsync(rssFeedUri);

async_op_with_progress.Progress(
    [](
        IAsyncOperationWithProgress<SyndicationFeed,
        RetrievalProgress> const& /* sender */,
        RetrievalProgress const& args)
    {
        uint32_t bytes_retrieved = args.BytesRetrieved;
        // use bytes_retrieved;
    });

async_op_with_progress.Completed(
    [](
        IAsyncOperationWithProgress<SyndicationFeed,
        RetrievalProgress> const& sender,
        AsyncStatus const /* asyncStatus */)
    {
        SyndicationFeed syndicationFeed = sender.GetResults();
        // use syndicationFeed;
    });

    // or (but this function must then be a coroutine, and return
    IAsyncAction)
    // SyndicationFeed syndicationFeed{ co_await async_op_with_progress };
}

```

As the "coroutine" comment above suggests, instead of using a delegate with the completed events of asynchronous actions and operations, you'll probably find it more natural to use coroutines. For details, and code examples, see [Concurrency and asynchronous operations with C++/WinRT](#).

ⓘ Note

It's not correct to implement more than one *completion handler* for an asynchronous action or operation. You can have either a single delegate for its completed event, or you can `co_await` it. If you have both, then the second will fail.

If you stick with delegates instead of a coroutine, then you can opt for a simpler syntax.

C++/WinRT

```

async_op_with_progress.Completed(
    [](auto&& /*sender*/, AsyncStatus const /* args */)
    {

```

```
// ...  
});
```

Delegate types that return a value

Some delegate types must themselves return a value. An example is [ListViewItemToKeyHandler](#), which returns a string. Here's an example of authoring a delegate of that type (note that the lambda function returns a value).

C++/WinRT

```
using namespace winrt::Windows::UI::Xaml::Controls;  
  
winrt::hstring f(ListView listview)  
{  
    return ListViewPersistenceHelper::GetRelativeScrollPosition(listview, []  
(IInspectable const& item)  
    {  
        return L"key for item goes here";  
    });  
}
```

Safely accessing the *this* pointer with an event-handling delegate

If you handle an event with an object's member function, or from within a lambda function inside an object's member function, then you need to think about the relative lifetimes of the event recipient (the object handling the event) and the event source (the object raising the event). For more info, and code examples, see [Strong and weak references in C++/WinRT](#).

Important APIs

- [winrt::auto_revoke_t](#) marker struct
- [winrt::implements::get_weak](#) function
- [winrt::implements::get_strong](#) function

Related topics

- [Author events in C++/WinRT](#)
- [Concurrency and asynchronous operations with C++/WinRT](#)

- Strong and weak references in C++/WinRT

Author events in C++/WinRT

Article • 03/16/2023

This topic builds on the Windows Runtime component, and the consuming application, that the [Windows Runtime components with C++/WinRT](#) topic shows you how to build.

Here are the new features that this topic adds.

- Update the thermometer runtime class to raise an event when its temperature goes below freezing.
- Update the Core App that consumes the thermometer runtime class so that it handles that event.

❗ Note

For info about installing and using the C++/WinRT Visual Studio Extension (VSIX) and the NuGet package (which together provide project template and build support), see [Visual Studio support for C++/WinRT](#).

📌 Important

For essential concepts and terms that support your understanding of how to consume and author runtime classes with C++/WinRT, see [Consume APIs with C++/WinRT](#) and [Author APIs with C++/WinRT](#).

Create ThermometerWRC and ThermometerCoreApp

If you want to follow along with the updates shown in this topic, so that you can build and run the code, then the first step is to follow the walkthrough in the [Windows Runtime components with C++/WinRT](#) topic. By doing so, you'll have the **ThermometerWRC** Windows Runtime component, and the **ThermometerCoreApp** Core App that consumes it.

Update ThermometerWRC to raise an event

Update `Thermometer.idl` to look like the listing below. This is how to declare an event whose delegate type is [EventHandler](#) with an argument of a single-precision floating-

point number.

idl

```
// Thermometer.idl
namespace ThermometerWRC
{
    runtimeclass Thermometer
    {
        Thermometer();
        void AdjustTemperature(Single deltaFahrenheit);
        event Windows.Foundation.EventHandler<Single>
        TemperatureIsBelowFreezing;
    };
}
```

Save the file. The project won't build to completion in its current state, but perform a build now in any case to generate updated versions of the

`\ThermometerWRC\ThermometerWRC\Generated Files\sources\Thermometer.h` and

`Thermometer.cpp` stub files. Inside those files you can now see stub implementations of the **TemperatureIsBelowFreezing** event. In C++/WinRT, an IDL-declared event is implemented as a set of overloaded functions (similar to the way a property is implemented as a pair of overloaded get and set functions). One overload takes a delegate to be registered, and returns a token (a [winrt::event_token](#)). The other takes a token, and revokes the registration of the associated delegate.

Now open `Thermometer.h` and `Thermometer.cpp`, and update the implementation of the **Thermometer** runtime class. In `Thermometer.h`, add the two overloaded **TemperatureIsBelowFreezing** functions, as well as a private event data member to use in the implementation of those functions.

C++/WinRT

```
// Thermometer.h
...
namespace winrt::ThermometerWRC::implementation
{
    struct Thermometer : ThermometerT<Thermometer>
    {
        ...
        winrt::event_token
        TemperatureIsBelowFreezing(Windows::Foundation::EventHandler<float> const&
        handler);
        void TemperatureIsBelowFreezing(winrt::event_token const& token)
        noexcept;

    private:
        winrt::event<Windows::Foundation::EventHandler<float>>
```

```

m_temperatureIsBelowFreezingEvent;
    ...
};
}
...

```

As you can see above, an event is represented by the `winrt::event` struct template, parameterized by a particular delegate type (which itself can be parameterized by an args type).

In `Thermometer.cpp`, implement the two overloaded **TemperatureIsBelowFreezing** functions.

C++/WinRT

```

// Thermometer.cpp
...
namespace winrt::ThermometerWRC::implementation
{
    winrt::event_token
    Thermometer::TemperatureIsBelowFreezing(Windows::Foundation::EventHandler<float> const& handler)
    {
        return m_temperatureIsBelowFreezingEvent.add(handler);
    }

    void Thermometer::TemperatureIsBelowFreezing(winrt::event_token const& token) noexcept
    {
        m_temperatureIsBelowFreezingEvent.remove(token);
    }

    void Thermometer::AdjustTemperature(float deltaFahrenheit)
    {
        m_temperatureFahrenheit += deltaFahrenheit;
        if (m_temperatureFahrenheit < 32.f)
            m_temperatureIsBelowFreezingEvent(*this, m_temperatureFahrenheit);
    }
}

```

❗ Note

For details of what an auto event revoker is, see **Revoke a registered delegate**. You get auto event revoker implementation for free for your event. In other words, you don't need to implement the overload for the event revoker—that's provided for you by the C++/WinRT projection.

The other overloads (the registration and manual revocation overloads) are *not* baked into the projection. That's to give you the flexibility to implement them optimally for your scenario. Calling `event::add` and `event::remove` as shown in these implementations is an efficient and concurrency/thread-safe default. But if you have a very large number of events, then you may not want an event field for each, but rather opt for some kind of sparse implementation instead.

You can also see above that the implementation of the **AdjustTemperature** function has been updated to raise the **TemperaturesBelowFreezing** event if the temperature goes below freezing.

Update ThermometerCoreApp to handle the event

In the **ThermometerCoreApp** project, in `App.cpp`, make the following changes to the code to register an event handler, and then cause the temperature to go below freezing.

`WINRT_ASSERT` is a macro definition, and it expands to `_ASSERTE`.

C++/WinRT

```
struct App : implements<App, IFrameworkViewSource, IFrameworkView>
{
    winrt::event_token m_eventToken;
    ...

    void Initialize(CoreApplicationView const &)
    {
        m_eventToken = m_thermometer.TemperatureIsBelowFreezing([](const
auto &, float temperatureFahrenheit)
        {
            WINRT_ASSERT(temperatureFahrenheit < 32.f); // Put a breakpoint
here.
        });
    }
    ...

    void Uninitialize()
    {
        m_thermometer.TemperatureIsBelowFreezing(m_eventToken);
    }
    ...

    void OnPointerPressed(IInspectable const &, PointerEventArgs const &
args)
    {
        m_thermometer.AdjustTemperature(-1.f);
        ...
    }
}
```

```
}  
...  
};
```

Be aware of the change to the **OnPointerPressed** method. Now, each time you click the window, you *subtract* 1 degree Fahrenheit from the thermometer's temperature. And now, the app is handling the event that's raised when the temperature goes below freezing. To demonstrate that the event is being raised as expected, put a breakpoint inside the lambda expression that's handling the **TemperaturesBelowFreezing** event, run the app, and click inside the window.

Parameterized delegates across an ABI

If your event must be accessible across an application binary interface (ABI)—such as between a component and its consuming application—then your event must use a Windows Runtime delegate type. The example above uses the [Windows::Foundation::EventHandler<T>](#) Windows Runtime delegate type. [TypedEventHandler<TSender, TResult>](#) is another example of a Windows Runtime delegate type.

The type parameters for those two delegate types have to cross the ABI, so the type parameters must be Windows Runtime types, too. That includes Windows runtime classes, third-party runtime classes, and primitive types such as numbers and strings. The compiler helps you with a "*T must be WinRT type*" error if you forget that constraint.

Below is an example in the form of code listings. Begin with the **ThermometerWRC** and **ThermometerCoreApp** projects that you created earlier in this topic, and edit the code in those projects to look like the code in these listings.

This first listing is for the **ThermometerWRC** project. After editing `ThermometerWRC.idl` as shown below, build the project, and then copy `MyEventArgs.h` and `.cpp` into the project (from the `Generated Files` folder) just like you did earlier with `Thermometer.h` and `.cpp`. Remember to delete the `static_assert` from both files.

C++/WinRT

```
// ThermometerWRC.idl  
namespace ThermometerWRC  
{  
    [default_interface]  
    runtimeclass MyEventArgs  
    {  
        Single TemperatureFahrenheit{ get; };  
    }  
}
```

```

[default_interface]
runtimeclass Thermometer
{
    ...
    event Windows.Foundation.EventHandler<ThermometerWRC.MyEventArgs>
TemperatureIsBelowFreezing;
    ...
};
}

// MyEventArgs.h
#pragma once
#include "MyEventArgs.g.h"

namespace winrt::ThermometerWRC::implementation
{
    struct MyEventArgs : MyEventArgsT<MyEventArgs>
    {
        MyEventArgs() = default;
        MyEventArgs(float temperatureFahrenheit);
        float TemperatureFahrenheit();

    private:
        float m_temperatureFahrenheit{ 0.f };
    };
}

// MyEventArgs.cpp
#include "pch.h"
#include "MyEventArgs.h"
#include "MyEventArgs.g.cpp"

namespace winrt::ThermometerWRC::implementation
{
    MyEventArgs::MyEventArgs(float temperatureFahrenheit) :
m_temperatureFahrenheit(temperatureFahrenheit)
    {
    }

    float MyEventArgs::TemperatureFahrenheit()
    {
        return m_temperatureFahrenheit;
    }
}

// Thermometer.h
...
struct Thermometer : ThermometerT<Thermometer>
{
    ...
    winrt::event_token
TemperatureIsBelowFreezing(Windows::Foundation::EventHandler<ThermometerWRC:
:MyEventArgs> const& handler);
    ...
}

```

```

private:

winrt::event<Windows::Foundation::EventHandler<ThermometerWRC::MyEventArgs>>
m_temperatureIsBelowFreezingEvent;
...
}
...

// Thermometer.cpp
#include "MyEventArgs.h"
...
winrt::event_token
Thermometer::TemperatureIsBelowFreezing(Windows::Foundation::EventHandler<ThermometerWRC::MyEventArgs> const& handler) { ... }
...
void Thermometer::AdjustTemperature(float deltaFahrenheit)
{
    m_temperatureFahrenheit += deltaFahrenheit;

    if (m_temperatureFahrenheit < 32.f)
    {
        auto args =
winrt::make_self<winrt::ThermometerWRC::implementation::MyEventArgs>
(m_temperatureFahrenheit);
        m_temperatureIsBelowFreezingEvent(*this, *args);
    }
}
...

```

This listing is for the **ThermometerCoreApplication** project.

C++/WinRT

```

// App.cpp
...
void Initialize(CoreApplicationView const&)
{
    m_eventToken = m_thermometer.TemperatureIsBelowFreezing([](const auto&,
ThermometerWRC::MyEventArgs args)
    {
        float degrees = args.TemperatureFahrenheit();
        WINRT_ASSERT(degrees < 32.f); // Put a breakpoint here.
    });
}
...

```

Simple signals across an ABI

If you don't need to pass any parameters or arguments with your event, then you can define your own simple Windows Runtime delegate type. The example below shows a

simpler version of the **Thermometer** runtime class. It declares a delegate type named **SignalDelegate** and then it uses that to raise a signal-type event instead of an event with a parameter.

idl

```
// ThermometerWRC.idl
namespace ThermometerWRC
{
    delegate void SignalDelegate();

    runtimeclass Thermometer
    {
        Thermometer();
        event ThermometerWRC.SignalDelegate
SignalTemperatureIsBelowFreezing;
        void AdjustTemperature(Single value);
    };
}
```

C++/WinRT

```
// Thermometer.h
...
namespace winrt::ThermometerWRC::implementation
{
    struct Thermometer : ThermometerT<Thermometer>
    {
        ...

        winrt::event_token
SignalTemperatureIsBelowFreezing(ThermometerWRC::SignalDelegate const&
handler);
        void SignalTemperatureIsBelowFreezing(winrt::event_token const&
token);
        void AdjustTemperature(float deltaFahrenheit);

    private:
        winrt::event<ThermometerWRC::SignalDelegate> m_signal;
        float m_temperatureFahrenheit{ 0.f };
    };
}
```

C++/WinRT

```
// Thermometer.cpp
...
namespace winrt::ThermometerWRC::implementation
{
    winrt::event_token
Thermometer::SignalTemperatureIsBelowFreezing(ThermometerWRC::SignalDelegate
```

```

const& handler)
{
    return m_signal.add(handler);
}

void Thermometer::SignalTemperatureIsBelowFreezing(winrt::event_token
const& token)
{
    m_signal.remove(token);
}

void Thermometer::AdjustTemperature(float deltaFahrenheit)
{
    m_temperatureFahrenheit += deltaFahrenheit;
    if (m_temperatureFahrenheit < 32.f)
    {
        m_signal();
    }
}
}

```

C++/WinRT

```

// App.cpp
struct App : implements<App, IFrameworkViewSource, IFrameworkView>
{
    ThermometerWRC::Thermometer m_thermometer;
    winrt::event_token m_eventToken;
    ...

    void Initialize(CoreApplicationView const &)
    {
        m_eventToken = m_thermometer.SignalTemperatureIsBelowFreezing([] {
/* ... */ });
    }
    ...

    void Uninitialize()
    {
        m_thermometer.SignalTemperatureIsBelowFreezing(m_eventToken);
    }
    ...

    void OnPointerPressed(IInspectable const &, PointerEventArgs const &
args)
    {
        m_thermometer.AdjustTemperature(-1.f);
        ...
    }
    ...
};

```

Parameterized delegates, simple signals, and callbacks within a project

If you need events that are internal to your Visual Studio project (not across binaries), where those events are not limited to Windows Runtime types, then you can still use the `winrt::event<Delegate>` class template. Simply use `winrt::delegate` instead of an actual Windows Runtime delegate type, since `winrt::delegate` also supports non Windows Runtime parameters.

The example below first shows a delegate signature that doesn't take any parameters (essentially a simple signal), and then one that takes a string.

C++/WinRT

```
winrt::event<winrt::delegate<>> signal;
signal.add([] { std::wcout << L"Hello, "; });
signal.add([] { std::wcout << L"World!" << std::endl; });
signal();

winrt::event<winrt::delegate<std::wstring>> log;
log.add([](std::wstring const& message) { std::wcout << message.c_str() <<
std::endl; });
log.add([](std::wstring const& message) { Persist(message); });
log(L"Hello, World!");
```

Notice how you can add to the event as many subscribing delegates as you wish. However, there is some overhead associated with an event. If all you need is a simple callback with only a single subscribing delegate, then you can use `winrt::delegate<... T>` on its own.

C++/WinRT

```
winrt::delegate<> signalCallback;
signalCallback = [] { std::wcout << L"Hello, World!" << std::endl; };
signalCallback();

winrt::delegate<std::wstring> logCallback;
logCallback = [](std::wstring const& message) { std::wcout <<
message.c_str() << std::endl; }f;
logCallback(L"Hello, World!");
```

If you're porting from a C++/CX codebase where events and delegates are used internally within a project, then `winrt::delegate` will help you to replicate that pattern in C++/WinRT.

Deferrable events

A common pattern in the Windows Runtime is the deferrable event. An event handler *takes a deferral* by calling the event argument's **GetDeferral** method. Doing so indicates to the event source that post-event activities should be postponed until the deferral is completed. This allows an event handler to perform asynchronous actions in response to an event.

The `winrt::deferrable_event_args` struct template is a helper class for implementing (producing) the Windows Runtime deferral pattern. Here's an example.

C++/WinRT

```
// Widget.idl
namespace Sample
{
    runtimeclass WidgetStartingEventArgs
    {
        Windows.Foundation.Deferral GetDeferral();
        Boolean Cancel;
    };

    runtimeclass Widget
    {
        event Windows.Foundation.TypedEventHandler<
            Widget, WidgetStartingEventArgs> Starting;
    };
}

// Widget.h
namespace winrt::Sample::implementation
{
    struct Widget : WidgetT<Widget>
    {
        Widget() = default;

        event_token Starting(Windows::Foundation::TypedEventHandler<
            Sample::Widget, Sample::WidgetStartingEventArgs> const& handler)
        {
            return m_starting.add(handler);
        }
        void Starting(event_token const& token) noexcept
        {
            m_starting.remove(token);
        }

    private:
        event<Windows::Foundation::TypedEventHandler<
            Sample::Widget, Sample::WidgetStartingEventArgs>> m_starting;
    };
}
```

```
struct WidgetStartingEventArgs :  
WidgetStartingEventArgsT<WidgetStartingEventArgs>,  
  
deferrable_event_args<WidgetStartingEventArgs>  
    //  
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  
    {  
        bool Cancel() const noexcept { return m_cancel; }  
        void Cancel(bool value) noexcept { m_cancel = value; }  
        bool m_cancel = false;  
    };  
};
```

Here's how the event recipient consumes the deferrable event pattern.

C++/WinRT

```
// EventRecipient.h
widget.Starting([](auto sender, auto args) -> fire_and_forget
{
    auto deferral = args.GetDeferral();
    if (!co_await CanWidgetStartAsync(sender))
    {
        // Do not allow the widget to start.
        args.Cancel(true);
    }
    deferral.Complete();
});
```

As the implementor (producer) of the event source, you derive your event args class from `winrt::deferrable_event_args`. `deferrable_event_args<T>` implements `T::GetDeferral` for you. It also exposes a new helper method `deferrable_event_args::wait_for_deferrals`, which completes when all outstanding deferrals have completed (if no deferrals were taken, then it completes immediately).

C++/WinRT

```
// Widget.h
IAsyncOperation<bool> TryStartWidget(Widget const& widget)
{
    auto args = make_self<WidgetStartingEventArgs>();
    // Raise the event to let people know that the widget is starting
    // and give them a chance to prevent it.
    m_starting(widget, *args);
    // Wait for deferrals to complete.
    co_await args->wait_for_deferrals();
    // Use the results.
    bool started = false;
    if (!args->Cancel())
    {
```

```
        widget.InsertBattery();  
        widget.FlipPowerSwitch();  
        started = true;  
    }  
    co_return started;  
}
```

Design guidelines

We recommend that you pass events, and not delegates, as function parameters. The **add** function of [winrt::event](#) is the one exception, because you must pass a delegate in that case. The reason for this guideline is because delegates can take different forms across different Windows Runtime languages (in terms of whether they support one client registration, or multiple). Events, with their multiple subscriber model, constitute a much more predictable and consistent option.

The signature for an event handler delegate should consist of two parameters: *sender* ([IInspectable](#)), and *args* (some event argument type, for example [RoutedEventArgs](#)).

Note that these guidelines don't necessarily apply if you're designing an internal API. Although, internal APIs often become public over time.

Related topics

- [Author APIs with C++/WinRT](#)
- [Consume APIs with C++/WinRT](#)
- [Handle events by using delegates in C++/WinRT](#)
- [Windows Runtime components with C++/WinRT](#)

Collections with C++/WinRT

Article • 10/20/2022

Internally, a Windows Runtime collection has a lot of complicated moving parts. But when you want to pass a collection object to a Windows Runtime function, or to implement your own collection properties and collection types, there are functions and base classes in [C++/WinRT](#) to support you. These features take the complexity out of your hands, and save you a lot of overhead in time and effort.

[IVector](#) is the Windows Runtime interface implemented by any random-access collection of elements. If you were to implement [IVector](#) yourself, you'd also need to implement [IIterable](#), [IVectorView](#), and [IIterator](#). Even if you *need* a custom collection type, that's a lot of work. But if you have data in a `std::vector` (or a `std::map`, or a `std::unordered_map`) and all you want to do is pass that to a Windows Runtime API, then you'd want to avoid doing that level of work, if possible. And avoiding it *is* possible, because C++/WinRT helps you to create collections efficiently and with little effort.

Also see [XAML items controls; bind to a C++/WinRT collection](#).

Helper functions for collections

General-purpose collection, empty

This section covers the scenario where you wish to create a collection that's initially empty; and then populate it *after* creation.

To retrieve a new object of a type that implements a general-purpose collection, you can call the [winrt::single_threaded_vector](#) function template. The object is returned as an [IVector](#), and that's the interface via which you call the returned object's functions and properties.

If you want to copy-paste the following code examples directly into the main source code file of a **Windows Console Application (C++/WinRT)** project, then first set **Not Using Precompiled Headers** in project properties.

C++/WinRT

```
// main.cpp
#include <winrt/Windows.Foundation.Collections.h>
#include <iostream>
using namespace winrt;
```

```

int main()
{
    winrt::init_apartment();

    Windows::Foundation::Collections::IVector<int> coll{
winrt::single_threaded_vector<int>() };
    coll.Append(1);
    coll.Append(2);
    coll.Append(3);

    for (auto const& el : coll)
    {
        std::cout << el << std::endl;
    }

    Windows::Foundation::Collections::IVectorView<int> view{ coll.GetView()
};
}

```

As you can see in the code example above, after creating the collection you can append elements, iterate over them, and generally treat the object as you would any Windows Runtime collection object that you might have received from an API. If you need an immutable view over the collection, then you can call [IVector::GetView](#), as shown. The pattern shown above—of creating and consuming a collection—is appropriate for simple scenarios where you want to pass data into, or get data out of, an API. You can pass an [IVector](#), or an [IVectorView](#), anywhere an [Iterable](#) is expected.

In the code example above, the call to `winrt::init_apartment` initializes the thread in the Windows Runtime; by default, in a multithreaded apartment. The call also initializes COM.

General-purpose collection, primed from data

This section covers the scenario where you wish to create a collection and populate it at the same time.

You can avoid the overhead of the calls to **Append** in the previous code example. You may already have the source data, or you may prefer to populate the source data in advance of creating the Windows Runtime collection object. Here's how to do that.

C++/WinRT

```

auto coll1{ winrt::single_threaded_vector<int>({ 1,2,3 }) };

std::vector<int> values{ 1,2,3 };
auto coll2{ winrt::single_threaded_vector<int>(std::move(values)) };

```

```
for (auto const& el : coll2)
{
    std::cout << el << std::endl;
}
```

You can pass a temporary object containing your data to `winrt::single_threaded_vector`, as with `coll1`, above. Or you can move a `std::vector` (assuming you won't be accessing it again) into the function. In both cases, you're passing an *rvalue* into the function. That enables the compiler to be efficient and to avoid copying the data. If you want to know more about *rvalues*, see [Value categories, and references to them](#).

If you want to bind a XAML items control to your collection, then you can. But be aware that to correctly set the `ItemsControl.ItemsSource` property, you need to set it to a value of type `IVector` of `IInspectable` (or of an interoperability type such as `IBindableObservableVector`).

Here's a code example that produces a collection of a type suitable for binding, and appends an element to it. You can find the context for this code example in [XAML items controls; bind to a C++/WinRT collection](#).

C++/WinRT

```
auto bookSkus{
    winrt::single_threaded_vector<Windows::Foundation::IInspectable>();
    bookSkus.Append(winrt::make<Bookstore::implementation::BookSku>(L"Moby
    Dick"));
}
```

You can create a Windows Runtime collection from data, and get a view on it ready to pass to an API, all without copying anything.

C++/WinRT

```
std::vector<float> values{ 0.1f, 0.2f, 0.3f };
Windows::Foundation::Collections::IVectorView<float> view{
    winrt::single_threaded_vector(std::move(values)).GetView() };
}
```

In the examples above, the collection we create *can* be bound to a XAML items control; but the collection isn't observable.

Observable collection

To retrieve a new object of a type that implements an *observable* collection, call the `winrt::single_threaded_observable_vector` function template with any element type. But

to make an observable collection suitable for binding to a XAML items control, use **IInspectable** as the element type.

The object is returned as an **IObservableVector**, and that's the interface via which you (or the control to which it's bound) call the returned object's functions and properties.

C++/WinRT

```
auto bookSkus{
    winrt::single_threaded_observable_vector<Windows::Foundation::IInspectable>
    () };
```

For more details, and code examples, about binding your user interface (UI) controls to an observable collection, see [XAML items controls; bind to a C++/WinRT collection](#).

Associative collection (map)

There are associative collection versions of the two functions that we've looked at.

- The **winrt::single_threaded_map** function template returns a non-observable associative collection as an **IMap**.
- The **winrt::single_threaded_observable_map** function template returns an observable associative collection as an **IObservableMap**.

You can optionally prime these collections with data by passing to the function an *rvalue* of type **std::map** or **std::unordered_map**.

C++/WinRT

```
auto coll1{
    winrt::single_threaded_map<winrt::hstring, int>(std::map<winrt::hstring,
int>{
    { L"AliceBlue", 0xffff0f8ff }, { L"AntiqueWhite", 0xffffaebd7 }
    })
};

std::map<winrt::hstring, int> values{
    { L"AliceBlue", 0xffff0f8ff }, { L"AntiqueWhite", 0xffffaebd7 }
};
auto coll2{ winrt::single_threaded_map<winrt::hstring, int>
    (std::move(values)) };
```

Single-threaded

The "single-threaded" in the names of these functions indicates that they don't provide any concurrency—in other words, they're not thread-safe. The mention of threads is unrelated to apartments, because the objects returned from these functions are all agile (see [Agile objects in C++/WinRT](#)). It's just that the objects are single-threaded. And that's entirely appropriate if you just want to pass data one way or the other across the application binary interface (ABI).

Base classes for collections

If, for complete flexibility, you want to implement your own custom collection, then you'll want to avoid doing that the hard way. For example, this is what a custom vector view would look like *without the assistance of C++/WinRT's base classes*.

C++/WinRT

```
...
using namespace winrt;
using namespace Windows::Foundation::Collections;
...
struct MyVectorView :
    implements<MyVectorView, IVectorView<float>, IIterable<float>>
{
    // IVectorView
    float GetAt(uint32_t const) { ... };
    uint32_t GetMany(uint32_t, winrt::array_view<float>) const { ... };
    bool IndexOf(float, uint32_t&) { ... };
    uint32_t Size() { ... };

    // IIterable
    IIterator<float> First() const { ... };
};
...
IVectorView<float> view{ winrt::make<MyVectorView>() };
```

Instead, it's much easier to derive your custom vector view from the [winrt::vector_view_base](#) struct template, and just implement the **get_container** function to expose the container holding your data.

C++/WinRT

```
struct MyVectorView2 :
    implements<MyVectorView2, IVectorView<float>, IIterable<float>>,
    winrt::vector_view_base<MyVectorView2, float>
{
    auto& get_container() const noexcept
    {
        return m_values;
    }
};
```

```

    }

private:
    std::vector<float> m_values{ 0.1f, 0.2f, 0.3f };
};

```

The container returned by `get_container` must provide the `begin` and `end` interface that `winrt::vector_view_base` expects. As shown in the example above, `std::vector` provides that. But you can return any container that fulfils the same contract, including your own custom container.

C++/WinRT

```

struct MyVectorView3 :
    implements<MyVectorView3, IVectorView<float>, IIterable<float>>,
    winrt::vector_view_base<MyVectorView3, float>
{
    auto get_container() const noexcept
    {
        struct container
        {
            float const* const first;
            float const* const last;

            auto begin() const noexcept
            {
                return first;
            }

            auto end() const noexcept
            {
                return last;
            }
        };

        return container{ m_values.data(), m_values.data() + m_values.size() };
    }

private:
    std::array<float, 3> m_values{ 0.2f, 0.3f, 0.4f };
};

```

These are the base classes that C++/WinRT provides to help you implement custom collections.

winrt::vector_view_base

See the code examples above.

winrt::vector_base

C++/WinRT

```
struct MyVector :
    implements<MyVector, IVector<float>, IVectorView<float>,
    IIterable<float>>>,
    winrt::vector_base<MyVector, float>
{
    auto& get_container() const noexcept
    {
        return m_values;
    }

    auto& get_container() noexcept
    {
        return m_values;
    }

private:
    std::vector<float> m_values{ 0.1f, 0.2f, 0.3f };
};
```

winrt::observable_vector_base

C++/WinRT

```
struct MyObservableVector :
    implements<MyObservableVector, IObservableVector<float>, IVector<float>,
    IVectorView<float>, IIterable<float>>>,
    winrt::observable_vector_base<MyObservableVector, float>
{
    auto& get_container() const noexcept
    {
        return m_values;
    }

    auto& get_container() noexcept
    {
        return m_values;
    }

private:
    std::vector<float> m_values{ 0.1f, 0.2f, 0.3f };
};
```

winrt::map_view_base

C++/WinRT

```
struct MyMapView :
    implements<MyMapView, IMapView<winrt::hstring, int>,
    IIterable<IKeyValuePair<winrt::hstring, int>>>,
    winrt::map_view_base<MyMapView, winrt::hstring, int>
{
    auto& get_container() const noexcept
    {
        return m_values;
    }

private:
    std::map<winrt::hstring, int> m_values{
        { L"AliceBlue", 0xffff0f8ff }, { L"AntiqueWhite", 0xfffaebd7 }
    };
};
```

winrt::map_base

C++/WinRT

```
struct MyMap :
    implements<MyMap, IMap<winrt::hstring, int>, IMapView<winrt::hstring,
int>, IIterable<IKeyValuePair<winrt::hstring, int>>>,
    winrt::map_base<MyMap, winrt::hstring, int>
{
    auto& get_container() const noexcept
    {
        return m_values;
    }

    auto& get_container() noexcept
    {
        return m_values;
    }

private:
    std::map<winrt::hstring, int> m_values{
        { L"AliceBlue", 0xffff0f8ff }, { L"AntiqueWhite", 0xfffaebd7 }
    };
};
```

winrt::observable_map_base

C++/WinRT

```
struct MyObservableMap :
    implements<MyObservableMap, IObservableMap<winrt::hstring, int>,
```

```

IMap<winrt::hstring, int>, IMapView<winrt::hstring, int>,
IIterable<KeyValuePair<winrt::hstring, int>>>,
    winrt::observable_map_base<MyObservableMap, winrt::hstring, int>
{
    auto& get_container() const noexcept
    {
        return m_values;
    }

    auto& get_container() noexcept
    {
        return m_values;
    }

private:
    std::map<winrt::hstring, int> m_values{
        { L"AliceBlue", 0xffff0f8ff }, { L"AntiqueWhite", 0xfffaebd7 }
    };
};

```

Important APIs

- [ItemsControl.ItemsSource](#) property
- [IObservableVector](#) interface
- [IVector](#) interface
- [winrt::map_base](#) struct template
- [winrt::map_view_base](#) struct template
- [winrt::observable_map_base](#) struct template
- [winrt::observable_vector_base](#) struct template
- [winrt::single_threaded_observable_map](#) function template
- [winrt::single_threaded_map](#) function template
- [winrt::single_threaded_observable_vector](#) function template
- [winrt::single_threaded_vector](#) function template
- [winrt::vector_base](#) struct template
- [winrt::vector_view_base](#) struct template

Related topics

- [Value categories, and references to them](#)
- [XAML items controls; bind to a C++/WinRT collection](#)

Concurrency and asynchronous operations with C++/WinRT

Article • 10/20/2022

Important

This topic introduces the concepts of *coroutines* and `co_await`, which we recommend that you use in both your UI *and* in your non-UI applications. For simplicity, most of the code examples in this introductory topic show **Windows Console Application (C++/WinRT)** projects. The later code examples in this topic do use coroutines, but for convenience the console application examples also continue to use the blocking `get` function call just before exiting, so that the application doesn't exit before finishing printing its output. You won't do that (call the blocking `get` function) from a UI thread. Instead, you'll use the `co_await` statement. The techniques that you'll use in your UI applications are described in the topic [Advanced concurrency and asynchrony](#).

This introductory topic shows some of the ways in which you can both create and consume Windows Runtime asynchronous objects with C++/WinRT. After reading this topic, especially for techniques you'll use in your UI applications, also see [Advanced concurrency and asynchrony](#).

Asynchronous operations and Windows Runtime "Async" functions

Any Windows Runtime API that has the potential to take more than 50 milliseconds to complete is implemented as an asynchronous function (with a name ending in "Async"). The implementation of an asynchronous function initiates the work on another thread, and returns immediately with an object that represents the asynchronous operation. When the asynchronous operation completes, that returned object contains any value that resulted from the work. The **Windows::Foundation** Windows Runtime namespace contains four types of asynchronous operation object.

- [IAsyncAction](#),
- [IAsyncActionWithProgress<TProgress>](#),
- [IAsyncOperation<TResult>](#), and
- [IAsyncOperationWithProgress<TResult, TProgress>](#).

Each of these asynchronous operation types is projected into a corresponding type in the **winrt::Windows::Foundation** C++/WinRT namespace. C++/WinRT also contains an internal await adapter struct. You don't use it directly but, thanks to that struct, you can write a `co_await` statement to cooperatively await the result of any function that returns one of these asynchronous operation types. And you can author your own coroutines that return these types.

An example of an asynchronous Windows function is [SyndicationClient::RetrieveFeedAsync](#), which returns an asynchronous operation object of type [IAsyncOperationWithProgress<TResult, TProgress>](#).

Let's look at some ways—first blocking, and then non-blocking—of using C++/WinRT to call an API such as that. Just for illustration of the basic ideas, we'll be using a **Windows Console Application (C++/WinRT)** project in the next few code examples. Techniques that are more appropriate for a UI application are discussed in [Advanced concurrency and asynchrony](#).

Block the calling thread

The code example below receives an asynchronous operation object from **RetrieveFeedAsync**, and it calls **get** on that object to block the calling thread until the results of the asynchronous operation are available.

If you want to copy-paste this example directly into the main source code file of a **Windows Console Application (C++/WinRT)** project, then first set **Not Using Precompiled Headers** in project properties.

C++/WinRT

```
// main.cpp
#include <winrt/Windows.Foundation.h>
#include <winrt/Windows.Web.Syndication.h>

using namespace winrt;
using namespace Windows::Foundation;
using namespace Windows::Web::Syndication;

void ProcessFeed()
{
    Uri rssFeedUri{ L"https://blogs.windows.com/feed" };
    SyndicationClient syndicationClient;
    SyndicationFeed syndicationFeed{
syndicationClient.RetrieveFeedAsync(rssFeedUri).get() };
    // use syndicationFeed.
}
```

```
int main()
{
    winrt::init_apartment();
    ProcessFeed();
}
```

Calling **get** makes for convenient coding, and it's ideal for console apps or background threads where you may not want to use a coroutine for whatever reason. But it's not concurrent nor asynchronous, so it's not appropriate for a UI thread (and an assertion will fire in unoptimized builds if you attempt to use it on one). To avoid holding up OS threads from doing other useful work, we need a different technique.

Write a coroutine

C++/WinRT integrates C++ coroutines into the programming model to provide a natural way to cooperatively wait for a result. You can produce your own Windows Runtime asynchronous operation by writing a coroutine. In the code example below, **ProcessFeedAsync** is the coroutine.

⚠ Note

The **get** function exists on the C++/WinRT projection type **winrt::Windows::Foundation::IAsyncAction**, so you can call the function from within any C++/WinRT project. You will not find the function listed as a member of the **IAsyncAction** interface, because **get** is not part of the application binary interface (ABI) surface of the actual Windows Runtime type **IAsyncAction**.

C++/WinRT

```
// main.cpp
#include <iostream>
#include <winrt/Windows.Foundation.Collections.h>
#include <winrt/Windows.Web.Syndication.h>

using namespace winrt;
using namespace Windows::Foundation;
using namespace Windows::Web::Syndication;

void PrintFeed(SyndicationFeed const& syndicationFeed)
{
    for (SyndicationItem const& syndicationItem : syndicationFeed.Items())
    {
        std::wcout << syndicationItem.Title().Text().c_str() << std::endl;
    }
}
```

```

IAsyncAction ProcessFeedAsync()
{
    Uri rssFeedUri{ L"https://blogs.windows.com/feed" };
    SyndicationClient syndicationClient;
    SyndicationFeed syndicationFeed{ co_await
syndicationClient.RetrieveFeedAsync(rssFeedUri) };
    PrintFeed(syndicationFeed);
}

int main()
{
    winrt::init_apartment();

    auto processOp{ ProcessFeedAsync() };
    // do other work while the feed is being printed.
    processOp.get(); // no more work to do; call get() so that we see the
printout before the application exits.
}

```

A coroutine is a function that can be suspended and resumed. In the **ProcessFeedAsync** coroutine above, when the `co_await` statement is reached, the coroutine asynchronously initiates the **RetrieveFeedAsync** call and then it immediately suspends itself and returns control back to the caller (which is **main** in the example above). **main** can then continue to do work while the feed is being retrieved and printed. When that's done (when the **RetrieveFeedAsync** call completes), the **ProcessFeedAsync** coroutine resumes at the next statement.

You can aggregate a coroutine into other coroutines. Or you can call **get** to block and wait for it to complete (and get the result if there is one). Or you can pass it to another programming language that supports the Windows Runtime.

It's also possible to handle the completed and/or progress events of asynchronous actions and operations by using delegates. For details, and code examples, see [Delegate types for asynchronous actions and operations](#).

As you can see, in the code example above, we continue to use the blocking **get** function call just before exiting **main**. But that's only so that the application doesn't exit before finishing printing its output.

Asynchronously return a Windows Runtime type

In this next example we wrap a call to **RetrieveFeedAsync**, for a specific URI, to give us a **RetrieveBlogFeedAsync** function that asynchronously returns a **SyndicationFeed**.

```

// main.cpp
#include <iostream>
#include <winrt/Windows.Foundation.Collections.h>
#include <winrt/Windows.Web.Syndication.h>

using namespace winrt;
using namespace Windows::Foundation;
using namespace Windows::Web::Syndication;

void PrintFeed(SyndicationFeed const& syndicationFeed)
{
    for (SyndicationItem const& syndicationItem : syndicationFeed.Items())
    {
        std::wcout << syndicationItem.Title().Text().c_str() << std::endl;
    }
}

IAsyncOperationWithProgress<SyndicationFeed, RetrievalProgress>
RetrieveBlogFeedAsync()
{
    Uri rssFeedUri{ L"https://blogs.windows.com/feed" };
    SyndicationClient syndicationClient;
    return syndicationClient.RetrieveFeedAsync(rssFeedUri);
}

int main()
{
    winrt::init_apartment();

    auto feedOp{ RetrieveBlogFeedAsync() };
    // do other work.
    PrintFeed(feedOp.get());
}

```

In the example above, **RetrieveBlogFeedAsync** returns an **IAsyncOperationWithProgress**, which has both progress and a return value. We can do other work while **RetrieveBlogFeedAsync** is doing its thing and retrieving the feed. Then, we call **get** on that asynchronous operation object to block, wait for it to complete, and then obtain the results of the operation.

If you're asynchronously returning a Windows Runtime type, then you should return an **IAsyncOperation<TResult>** or an **IAsyncOperationWithProgress<TResult, TProgress>**. Any first- or third-party runtime class qualifies, or any type that can be passed to or from a Windows Runtime function (for example, `int`, or `winrt::hstring`). The compiler will help you with a "*T must be WinRT type*" error if you try to use one of these asynchronous operation types with a non-Windows Runtime type.

If a coroutine doesn't have at least one `co_await` statement then, in order to qualify as a coroutine, it must have at least one `co_return` or one `co_yield` statement. There will be cases where your coroutine can return a value without introducing any asynchrony, and therefore without blocking nor switching context. Here's an example that does that (the second and subsequent times it's called) by caching a value.

C++/WinRT

```
winrt::hstring m_cache;

IAsyncOperation<winrt::hstring> ReadAsync()
{
    if (m_cache.empty())
    {
        // Asynchronously download and cache the string.
    }
    co_return m_cache;
}
```

Asynchronously return a non-Windows- Runtime type

If you're asynchronously returning a type that's *not* a Windows Runtime type, then you should return a Parallel Patterns Library (PPL) `concurrency::task`. We recommend `concurrency::task` because it gives you better performance (and better compatibility going forward) than `std::future` does.

Tip

If you include `<pplawait.h>`, then you can use `concurrency::task` as a coroutine type.

C++/WinRT

```
// main.cpp
#include <iostream>
#include <ppltasks.h>
#include <winrt/Windows.Foundation.Collections.h>
#include <winrt/Windows.Web.Syndication.h>

using namespace winrt;
using namespace Windows::Foundation;
using namespace Windows::Web::Syndication;

concurrency::task<std::wstring> RetrieveFirstTitleAsync()
```

```

{
    return concurrency::create_task([]
    {
        Uri rssFeedUri{ L"https://blogs.windows.com/feed" };
        SyndicationClient syndicationClient;
        SyndicationFeed syndicationFeed{
syndicationClient.RetrieveFeedAsync(rssFeedUri).get() };
        return std::wstring{
syndicationFeed.Items().GetAt(0).Title().Text() };
    });
}

int main()
{
    winrt::init_apartment();

    auto firstTitleOp{ RetrieveFirstTitleAsync() };
    // Do other work here.
    std::wcout << firstTitleOp.get() << std::endl;
}

```

Parameter-passing

For synchronous functions, you should use `const&` parameters by default. That will avoid the overhead of copies (which involve reference counting, and that means interlocked increments and decrements).

C++/WinRT

```

// Synchronous function.
void DoWork(Param const& value);

```

But you can run into problems if you pass a reference parameter to a coroutine.

C++/WinRT

```

// NOT the recommended way to pass a value to a coroutine!
IAsyncAction DoWorkAsync(Param const& value)
{
    // While it's ok to access value here...

    co_await DoOtherWorkAsync(); // (this is the first suspension point)...

    // ...accessing value here carries no guarantees of safety.
}

```

In a coroutine, execution is synchronous up until the first suspension point, where control is returned to the caller and the calling frame goes out of scope. By the time the

coroutine resumes, anything might have happened to the source value that a reference parameter references. From the coroutine's perspective, a reference parameter has uncontrolled lifetime. So, in the example above, we're safe to access *value* up until the `co_await`, but not after it. In the event that *value* is destructed by the caller, attempting to access it inside the coroutine after that results in a memory corruption. Nor can we safely pass *value* to **DoOtherWorkAsync** if there's any risk that that function will in turn suspend and then try to use *value* after it resumes.

To make parameters safe to use after suspending and resuming, your coroutines should use pass-by-value by default to ensure that they capture by value, and avoid lifetime issues. Cases when you can deviate from that guidance because you're certain that it's safe to do so are going to be rare.

C++/WinRT

```
// Coroutine
IASyncAction DoWorkAsync(Param value); // not const&
```

Passing by value requires that the argument be inexpensive to move or copy; and that's typically the case for a smart pointer.

It's also arguable that (unless you want to move the value) passing by const value is good practice. It won't have any effect on the source value from which you're making a copy, but it makes the intent clear, and helps if you inadvertently modify the copy.

C++/WinRT

```
// coroutine with strictly unnecessary const (but arguably good practice).
IASyncAction DoWorkAsync(Param const value);
```

Also see [Standard arrays and vectors](#), which deals with how to pass a standard vector into an asynchronous callee.

If you can't change your coroutine's signature, but you can change the implementation, then you can make a local copy before the first `co_await`.

C++/WinRT

```
IASyncAction DoWorkAsync(Param const& value)
{
    auto safe_value = value;
    // It's ok to access both safe_value and value here.

    co_await DoOtherWorkAsync();
}
```

```
    // It's ok to access only safe_value here (not value).  
}
```

If `Param` is expensive to copy, then extract just the pieces you need before the first `co_await`.

C++/WinRT

```
IASyncAction DoWorkAsync(Param const& value)  
{  
    auto safe_data = value.data;  
    // It's ok to access safe_data, value.data, and value here.  
  
    co_await DoOtherWorkAsync();  
  
    // It's ok to access only safe_data here (not value.data, nor value).  
}
```

Safely accessing the *this* pointer in a class-member coroutine

See [Strong and weak references in C++/WinRT](#).

Important APIs

- [concurrency::task](#) class
- [IAsyncAction](#) interface
- [IAsyncActionWithProgress<TProgress>](#) interface
- [IAsyncOperation<TResult>](#) interface
- [IAsyncOperationWithProgress<TResult, TProgress>](#) interface
- [SyndicationClient::RetrieveFeedAsync](#) method
- [SyndicationFeed](#) class

Related topics

- [Advanced concurrency and asynchrony](#)
- [Handle events by using delegates in C++/WinRT](#)
- [Standard C++ data types and C++/WinRT](#)

Advanced concurrency and asynchrony with C++/WinRT

Article • 10/20/2022

This topic describes advanced scenarios with concurrency and asynchrony in C++/WinRT.

For an introduction to this subject, first read [Concurrency and asynchronous operations](#).

Offloading work onto the Windows thread pool

A coroutine is a function like any other in that a caller is blocked until a function returns execution to it. And, the first opportunity for a coroutine to return is the first `co_await`, `co_return`, or `co_yield`.

So, before you do compute-bound work in a coroutine, you need to return execution to the caller (in other words, introduce a suspension point) so that the caller isn't blocked. If you're not already doing that by `co_await`-ing some other operation, then you can `co_await` the [winrt::resume_background](#) function. That returns control to the caller, and then immediately resumes execution on a thread pool thread.

The thread pool being used in the implementation is the low-level [Windows thread pool](#), so it's optimally efficient.

C++/WinRT

```
IAsyncOperation<uint32_t> DoWorkOnThreadPoolAsync()
{
    co_await winrt::resume_background(); // Return control; resume on thread pool.

    uint32_t result;
    for (uint32_t y = 0; y < height; ++y)
        for (uint32_t x = 0; x < width; ++x)
        {
            // Do compute-bound work here.
        }
    co_return result;
}
```

Programming with thread affinity in mind

This scenario expands on the previous one. You offload some work onto the thread pool, but then you want to display progress in the user interface (UI).

C++/WinRT

```
IAsyncAction DoWorkAsync(TextBlock textblock)
{
    co_await winrt::resume_background();
    // Do compute-bound work here.

    textblock.Text(L"Done!"); // Error: TextBlock has thread affinity.
}
```

The code above throws a [winrt::hresult_wrong_thread](#) exception, because a **TextBlock** must be updated from the thread that created it, which is the UI thread. One solution is to capture the thread context within which our coroutine was originally called. To do that, instantiate a [winrt::apartment_context](#) object, do background work, and then `co_await` the `apartment_context` to switch back to the calling context.

C++/WinRT

```
IAsyncAction DoWorkAsync(TextBlock textblock)
{
    winrt::apartment_context ui_thread; // Capture calling context.

    co_await winrt::resume_background();
    // Do compute-bound work here.

    co_await ui_thread; // Switch back to calling context.

    textblock.Text(L"Done!"); // Ok if we really were called from the UI
    thread.
}
```

As long as the coroutine above is called from the UI thread that created the **TextBlock**, then this technique works. There will be many cases in your app where you're certain of that.

For a more general solution to updating UI, which covers cases where you're uncertain about the calling thread, you can `co_await` the [winrt::resume_foreground](#) function to switch to a specific foreground thread. In the code example below, we specify the foreground thread by passing the dispatcher object associated with the **TextBlock** (by accessing its [Dispatcher](#) property). The implementation of `winrt::resume_foreground` calls [CoreDispatcher.RunAsync](#) on that dispatcher object to execute the work that comes after it in the coroutine.

```

IAsyncAction DoWorkAsync(TextBlock textblock)
{
    co_await winrt::resume_background();
    // Do compute-bound work here.

    // Switch to the foreground thread associated with textblock.
    co_await winrt::resume_foreground(textblock.Dispatcher());

    textblock.Text(L"Done!"); // Guaranteed to work.
}

```

The `winrt::resume_foreground` function takes an optional priority parameter. If you're using that parameter, then the pattern shown above is appropriate. If not, then you can choose to simplify `co_await winrt::resume_foreground(someDispatcherObject);` into just `co_await someDispatcherObject;`

Execution contexts, resuming, and switching in a coroutine

Broadly speaking, after a suspension point in a coroutine, the original thread of execution may go away and resumption may occur on any thread (in other words, any thread may call the **Completed** method for the async operation).

But if you `co_await` any of the four Windows Runtime asynchronous operation types (**IAsyncXxx**), then C++/WinRT captures the calling context at the point you `co_await`. And it ensures that you're still on that context when the continuation resumes. C++/WinRT does this by checking whether you're already on the calling context and, if not, switching to it. If you were on a single-threaded apartment (STA) thread before `co_await`, then you'll be on the same one afterward; if you were on a multi-threaded apartment (MTA) thread before `co_await`, then you'll be on one afterward.

```

IAsyncAction ProcessFeedAsync()
{
    Uri rssFeedUri{ L"https://blogs.windows.com/feed" };
    SyndicationClient syndicationClient;

    // The thread context at this point is captured...
    SyndicationFeed syndicationFeed{ co_await
    syndicationClient.RetrieveFeedAsync(rssFeedUri) };
}

```

```
    // ...and is restored at this point.  
}
```

The reason you can rely on this behavior is because C++/WinRT provides code to adapt those Windows Runtime asynchronous operation types to the C++ coroutine language support (these pieces of code are called wait adapters). The remaining awaitable types in C++/WinRT are simply thread pool wrappers and/or helpers; so they complete on the thread pool.

C++/WinRT

```
using namespace std::chrono_literals;  
IAsyncOperation<int> return_123_after_5s()  
{  
    // No matter what the thread context is at this point...  
    co_await 5s;  
    // ...we're on the thread pool at this point.  
    co_return 123;  
}
```

If you `co_await` some other type—even within a C++/WinRT coroutine implementation—then another library provides the adapters, and you'll need to understand what those adapters do in terms of resumption and contexts.

To keep context switches down to a minimum, you can use some of the techniques that we've already seen in this topic. Let's see some illustrations of doing that. In this next pseudo-code example, we show the outline of an event handler that calls a Windows Runtime API to load an image, drops onto a background thread to process that image, and then returns to the UI thread to display the image in the UI.

C++/WinRT

```
IAsyncAction MainPage::ClickHandler(IInspectable /* sender */,  
    RoutedEventArgs /* args */)   
{  
    // We begin in the UI context.  
  
    // Call StorageFile::OpenAsync to load an image file.  
  
    // The call to OpenAsync occurred on a background thread, but C++/WinRT  
    // has restored us to the UI thread by this point.  
  
    co_await winrt::resume_background();  
  
    // We're now on a background thread.  
  
    // Process the image.
```

```

co_await winrt::resume_foreground(this->Dispatcher());

// We're back on MainPage's UI thread.

// Display the image in the UI.
}

```

For this scenario, there's a little bit of inefficiency around the call to **StorageFile::OpenAsync**. There's a necessary context switch to a background thread (so that the handler can return execution to the caller), on resumption after which C++/WinRT restores the UI thread context. But, in this case, it's not necessary to be on the UI thread until we're about to update the UI. The more Windows Runtime APIs we call *before* our call to **winrt::resume_background**, the more unnecessary back-and-forth context switches we incur. The solution is not to call *any* Windows Runtime APIs before then. Move them all after the **winrt::resume_background**.

C++/WinRT

```

IAsyncAction MainPage::ClickHandler(IInspectable /* sender */,
RoutedEventArgs /* args */)
{
    // We begin in the UI context.

    co_await winrt::resume_background();

    // We're now on a background thread.

    // Call StorageFile::OpenAsync to load an image file.

    // Process the image.

    co_await winrt::resume_foreground(this->Dispatcher());

    // We're back on MainPage's UI thread.

    // Display the image in the UI.
}

```

If you want to do something more advanced, then you could write your own await adapters. For example, if you want a `co_await` to resume on the same thread that the async action completes on (so, there's no context switch), then you could begin by writing await adapters similar to the ones shown below.

ⓘ Note

The code example below is provided for educational purposes only; it's to get you started understanding how await adapters work. If you want to use this technique

in your own codebase, then we recommend that you develop and test your own await adapter struct(s). For example, you could write **complete_on_any**, **complete_on_current**, and **complete_on(dispatcher)**. Also consider making them templates that take the **IAsyncXxx** type as a template parameter.

C++/WinRT

```
struct no_switch
{
    no_switch(Windows::Foundation::IAsyncAction const& async) :
    m_async(async)
    {
    }

    bool await_ready() const
    {
        return m_async.Status() ==
        Windows::Foundation::AsyncStatus::Completed;
    }

    void await_suspend(std::experimental::coroutine_handle<> handle) const
    {
        m_async.Completed([handle](Windows::Foundation::IAsyncAction const&
        /* asyncInfo */, Windows::Foundation::AsyncStatus const& /* asyncStatus */)
        {
            handle();
        });
    }

    auto await_resume() const
    {
        return m_async.GetResults();
    }

private:
    Windows::Foundation::IAsyncAction const& m_async;
};
```

To understand how to use the **no_switch** await adapters, you'll first need to know that when the C++ compiler encounters a **co_await** expression it looks for functions called **await_ready**, **await_suspend**, and **await_resume**. The C++/WinRT library provides those functions so that you get reasonable behavior by default, like this.

C++/WinRT

```
IAsyncAction async{ ProcessFeedAsync() };
co_await async;
```